Uniwersytet Rzeszowski

Kolegium Nauk Przyrodniczych

Imię i nazwisko:

Piotr Nowak 125147

Seweryn Mik 127767

Przedmiot:

Pakiety obliczeń matematycznych i inżynierskich

Tytuł projekty:

Schemat Huffmana

Prowadzący:

dr hab. prof. UR Marcin Wesołowski

Wydział:

Informatyka

I rok, studia inżynierskie pierwszego stopnia, stacjonarne

Data:

II semestr 2023, Rzeszów

Temat projektu:

Celem programu jest zbudowanie schematu Huffmana dla danego łańcucha znaków oraz wypisanie dla komunikatu kodu jednoznacznego.

Opis zmiennych:

tabAll - Jest to lista, która przechowuje wszystkie etapy drzewa Huffmana. Każdy etap to lista par znaków i ich częstości wystąpień.
charCounts - Jest to lista par znaków i odpowiadających im częstości wystąpień w łańcuchu znaków.
lenWord - Przechowuje długość łańcucha znaków.
lenTab - Przechowuje aktualną liczbę par znaków w liście charCounts.
{min1, min2} - Pary znaków o najmniejszych częstościach wystąpień w liście charCounts.
newElem - Lista zawierająca parę znaków utworzoną przez połączenie min1 i min2 oraz sumę ich częstości wystąpień.
restElem - Lista zawierająca wszystkie pozostałe pary znaków z charCounts poza min1 i min2.
suma - Zmienna pomocnicza przechowująca sumę częstości wystąpień znaków.
end - Zmienna pomocnicza używana do kontrolowania indeksów w pętli for.
tabTree - Lista przechowująca pary znaków i ich położenie w drzewie Huffmana.
tabLett - Jest to lista, która będzie przechowywać pary (znak, bit) dla każdego węzła w strukturze Huffmana.
tabBin - Jest to lista, która będzie przechowywać pary (znak, bit) w postaci listy, przygotowanej do utworzenia drzewa Huffmana.
result - Jest to lista, która będzie przechowywać pary (znak, kod) dla każdego znaku w strukturze Huffmana.

Opis funkcji:

Sprawdzane jest, czy lista charCounts zawiera co najmniej 2 elementy. Jeśli nie, zwracana jest oryginalna lista charCounts. Jeśli lista charCounts ma co najmniej 2 elementy, sortowana jest ona według wartości drugiego elementu (czyli częstości wystąpień). Do zmiennych min1 i min2 przypisywane są pierwsze dwie pary znaków z posortowanej listy. Tworzona jest nowa zmienna newElem, zawierająca jedną parę znaków, której pierwszy element to połączenie znaków z min1 i min2, a drugi element to suma częstości wystąpień z min1 i min2. Do zmiennej restElem przypisywana jest lista charCounts, z której usunięte są pary min1 i min2. Wynikiem funkcji jest lista, która zawiera newElem na początku, a następnie restElem (pozostałe pary znaków).

Kod:

```
addTwoSmallest[tab_] :=
 Module[{min1, min2, newElem, restElem, suma = 0},
 If[Length[tab] < 2,
 Return[tab], {min1, min2} = SortBy[tab, Last][[1 ;; 2]];
 newElem = {{StringJoin[min1[[1]], min2[[1]]],
 min1[[2]] + min2[[2]]}};
 restElem = DeleteCases[tab, min1 | min2];
 Join[newElem, restElem]]];
wordbase =
 InputString[
 "Wprowadź łańcuch znaków: "]; (*Wyświetlenie okna w którym \
użytkownik wpisze ciąg znaków*)
word = ToLowerCase[
 StringReplace[wordbase,
 " " -> "_"]]; (*Operacja na łańcuchu:zmiana wszystkich liter na \
małe[ToLowerCase] i zastąpienie spacji zakiem "_" [StringReplace]*)
lenWord = StringLength[word];
tabAll = {};
If[lenWord > 1,
charCounts =
 Tally[Characters[
 word]]; (*Warunek - wartość wprowadzona musi być dłuższa od 1*)
lenTab = Length[charCounts];
For[i = 1, i <= Length[charCounts], i++,
 charCounts[[i, 2]] =
 charCounts[[i, 2]]/
 lenWord;]; (*Pętla, która zamienia wartości stałe na ułamkowe \
np: {b,3} -> {b,3/5}*)
charCounts =
 SortBy[charCounts,
 Last]; (*Sortowanie tablicy od najmniejszej wartości do \
największej*)
tabAll = {charCounts};
While[lenTab > 1, charCounts = addTwoSmallest[charCounts];
 PrependTo[tabAll, charCounts];
 lenTab--;]; (*Pętla while, która wywołuje funkcję addTwoSmallest i \
nadpisuje tablice charCounts, dodając ją potem na początek \
[PrependTo] tablicy tabAll*)
tabAll =
 Flatten[tabAll,
 1]; (*Usuwanie zagnieżdżeń w tablicy np: \
{{{b,1/4},{n,1/2}},{a,1/4}}} -> {{b,1/4},{n,1/2},{a,1/4}}*)
tabAll =
 Map[First,
 tabAll, {1}];(*Nadpisanie tablicy tabAll tak aby znajdowały się w \
niej litery np: {{b,1/4},{n,1/2},{a,1/4}} -> {b,n,a}*)
tabAll = DeleteDuplicates[tabAll]; (*Usuwanie duplikatów z tablicy*)
lenTab = Length[tabAll];
end = 0;
tabTree = {};
tabLett = {};
For[i = 1, i < lenTab, i++,
 If[StringLength[tabAll[[i]]] > 1,
 If[i*2 - end <= lenTab,
 AppendTo[tabTree, {tabAll[[i]] -> tabAll[[i*2 - end]], 1}];
 AppendTo[
 tabLett, {tabAll[[i*2 - end]],
 1}] (*Dodawanie do tablicy tabLett 1*)
 ];
```

```
 If[i*2 + 1 - end <= lenTab,
 AppendTo[tabTree, {tabAll[[i]] -> tabAll[[i*2 + 1 - end]], 0}];
 AppendTo[
 tabLett, {tabAll[[i*2 + 1 - end]],
 0}] (*Dodawanie do tablicy tabLett 1*)
 ];,
 end = end + 2;
 ];
 ];
tabBin = {};
For[i = 1, i <= Length[tabLett], i++,
 AppendTo[tabBin,
 Thread[{Characters[ToString[tabLett[[i, 1]]]], tabLett[[i, 2]]}]] (*
 Dodawania do tablicy tabBin rozdzielonych [
 Characters] znaków oraz łączy listę pojedynczych znaków [
 Thread] np: {{a,b},0}-> {{a,0},{b,0}}*)
 ];
tabBin =
 GroupBy[Flatten[tabBin, 1],
 First];(*Grupuje tablice od pierwszego wyrazu, dodatkowo \
upraszcza/usuwa zagnieżdżenia [Flatten]*)
result = {};
For[i = 1, i <= Length[tabBin], i++,
 results =
 GroupBy[tabBin[[i]], First -> Last,
 StringJoin @@ ToString /@ # &];(*Grupuje wszystkie wyrazy,
 zmienia wartości cyfrowe na tekstowe [ToString] oraz łączy je [
 StringJoin], dzięki temu łącząc 1,1,1 nie dostaniemy 3 tylko 111*)
 AppendTo[result,
 KeyValueMap[List,
 results]]]; (*dodaje przekształconą listę par klucz-wartość do \
listy result*)
result = Flatten[result, 1];
Print[Labeled[
 Framed[LayeredGraphPlot[tabTree,
 VertexLabels(*VertexLabeling*)-> Automatic,
 DirectedEdges -> True]], {"Schemat Huffmana dla ciągu znaków: "/
 wordbase}, {Top}]];(*Funckja określająca kolejność strzałek na \
wykresie*)
For[i = 1, i <= Length[result], i++,
 Print[result[[i, 1]], " - ",
 result[[i,
 2]]] (*Wypisywanie kodu jednoznacznego dla każdej wartości*)
 ];,
Print["Wprowadziłeś zbyt małą ilość znaków, aby stworzyć schemat \
Huffmana!"];
Abort[];](*Walidacja na wypadek gdyby ktoś wprowadził wartość <2*)
```